

CatHolyc

Uma linguagem de programação bíblica que vira Python

Trabalho Prático 1 · Linguagens Formais e Compiladores · UNIJUÍ

Como o transpilador funciona



Erros com linha

Léxico, sintático e semântico informam a linha do erro.

Fases separadas

A gramática valida a forma; a semântica valida o contexto.

Tradução de verdade

O Python sai da AST, não de troca de palavras.

O vocabulário

Tipos e declarações

capítulo → int
versículo → float
salmo → str
dogma → bool
mandamento → constante
preceito → variável (\$nome)

Decisão e repetição

caso → if
doutra_sorte_caso → elif
doutra_sorte → else
enquanto → while
peregrine → for
em_cada → for each

Operadores

somado_a → +
privado_de → -
multiplicado_por → *
partilhado_entre → /
conforme_a → ==
em_comunhão_com → and

Funções, laços e E/S

oficio → def
retorne → return
cessai → break
perseverai → continue
proclame → print
confesse → input

O que o gerador decide

Indentação por pilha

amém_senhor só desempilha um nível e não gera texto. Um bloco vazio vira pass.

Constantes e padrões

mandamento vira nome em MAIÚSCULAS. preceito sem valor recebe 0, 0.0, texto vazio ou False.

Divisão

capítulo ÷ capítulo vira //, senão /. Com negativos difere do C (trunca), por isso há aviso e a opção --strict.

Escopo de bloco

Python não tem escopo de bloco. Uma variável que sombreia outra ganha sufixo, como x__1.

peregrine

Vira a inicialização seguida de um while, com o incremento no fim do corpo.

Entrada de dados

confesse converte pelo tipo (int, float ou texto). Um dogma aceita verdade, true, 1 ou sim.

Decisão: do CatHolyc para o Python

Programa CatHolyc (caso.holy)

```
em_nome_do_pai
mandamento capítulo $maioridade seja 18 amém
preceito capítulo $idade seja 21 amém
preceito dogma $estudante seja verdade amém
caso $idade exaltado_sobre $maioridade em_comunhão_com $estudante
conforme_a verdade
proclame "É maior de idade e estudante." amém
doutra_sorte
proclame "A condição não foi satisfeita." amém
amém_senhor
assim_seja_em_seu_nome_amem
```

Python gerado

```
# em_nome_do_pai
MAIORIDADE = 18
idade = 21
estudante = True
if idade > MAIORIDADE and estudante == True:
    print("É maior de idade e estudante.")
else:
    print("A condição não foi satisfeita.")
```

Saída: É maior de idade e estudante. A constante vira MAIORIDADE; em_comunhão_com vira and.

Funções

Programa CatHolyc (funcao_soma.holy)

```
em_nome_do_pai
oficio capítulo $soma capítulo $a capítulo $b
  retorne $a somado_a $b amém
amém_senhor
oficio capítulo $fatorial capítulo $n
  preceito capítulo $r seja 1 amém
  preceito capítulo $i seja 1 amém
  enquanto $i não_acima_de $n
    $r seja $r multiplicado_por $i amém
    $i seja $i somado_a 1 amém
  amém_senhor
  retorne $r amém
amém_senhor
proclame $soma(2, 3) amém
proclame $fatorial(5) amém
assim_seja_em_seu_nome_amem
```

Python gerado

```
# em_nome_do_pai
def soma(a, b):
    return a + b
def fatorial(n):
    r = 1
    i = 1
    while i <= n:
        r = r * i
        i = i + 1
    return r
print(soma(2, 3))
print(fatorial(5))
```

Saída: 5 (soma de 2 e 3) e 120 (fatorial de 5). Cada oficio vira uma def.

Decisões próprias de projeto

1

Operadores só com palavras

Não há símbolos como + ou ==.
Tudo é palavra reservada:
somado_a, conforme_a,
em_comunhão_com.

2

Prefixo \$ e dois fechamentos

Variáveis levam \$ (estilo PHP):
\$idade. amém fecha comando;
amém_senhor fecha bloco.
Condições sem parênteses.

3

Funções e laços próprios

oficio e retorne definem funções,
chamadas como \$soma(2, 3).
peregrine usa atribuições no
cabeçalho; cessai e perseverai são
break e continue.

Extra: não_seja nega a comparação inteira: não_seja \$a exaltado_sobre \$b equivale a not ($a > b$).

Léxico e gramática

Análise léxica

- Identificadores sempre começam com \$: \$idade. Palavra sem \$ é erro léxico
- Maior casamento: lê a palavra inteira e só depois consulta a tabela de reservadas
- amém ≠ amém_senhor
- doutra_sorte ≠ doutra_sorte_caso
- Comentários: glosa(texto) ou |> texto <|
- Léxico acumula todos os erros; parser e semântica param no primeiro

Precedência (menor → maior)

ou_porventura

em_comunhão_com

não_seja

relacionais (conforme_a, exaltado_sobre...)

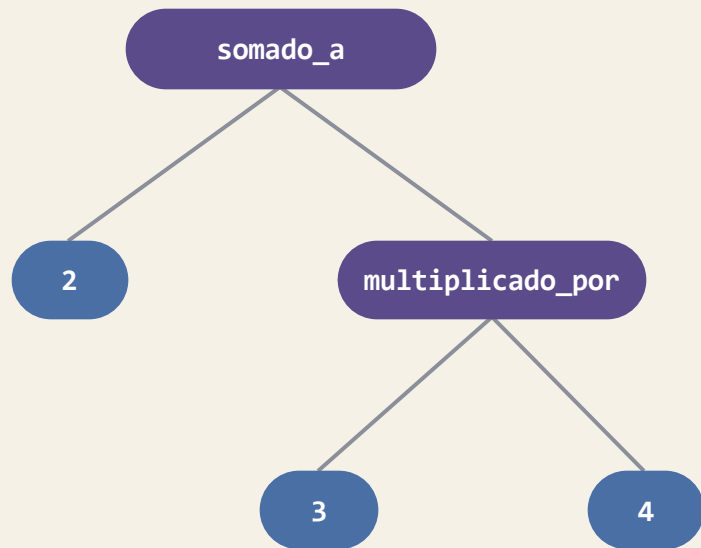
somado_a · privado_de

multiplicado_por · partilhado_entre · dízimo_de

privado_de unário · ()

AST e análise semântica

2 somado_a 3 multiplicado_por 4



Verificações semânticas

- Variável usada sem declaração
- Redecaração no mesmo escopo
- Tipos incompatíveis em atribuições e operações
- mandamento (constante) não pode ser alterado
- Condições de caso e enquanto devem ser dogma
- cessai, retorne e chamadas só em contexto válido

A análise para no primeiro erro, e nenhum código é gerado.

Testes e demonstração

hello	saída básica	Olá, mundo.
vars_const	constante e os 4 tipos	21 · 1.75 · Maria · True · 18
caso	caso, doutra_sorte, and	É maior de idade e estudante.
enquanto_countdown	enquanto	3 · 2 · 1
peregrine_sum	peregrine (soma de 1 a 5)	15
laco_cessai	cessai e perseverai	1 · 3 · 4
funcao_soma	oficio, retorne e chamadas	5 · 120

Cada programa tem o Python esperado e a saída esperada, conferidos automaticamente. Erros léxicos, sintáticos e semânticos são cobertos pelos testes automatizados de cada fase.

```
py cli.py tests\programs\caso.holy --run · py -m pytest -q → 128 testes
```